

Protect Agent Messages - HW 2 Report

Author: Zaahir Sharma

Course: CSCE 465

Date: October 4, 2026

Task 0 - Lab Preparation

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ python3 --version
Python 3.12.3
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ python -m pip show cryptography pytest
Name: cryptography
Version: 49.0.0
Summary: cryptography is a package which provides cryptographic recipes and primitives to Python developers
Home-page: https://github.com/pyca/cryptography
Author:
Author-email: The Python Cryptographic Authority and individual contributors <cryptography-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:
---
Name: pytest
Version: 9.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laughner, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel, Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$
```

Task 1 - Why is Encryption Alone Insufficient?

My Design

I built a local sender, relay, and receiver simulation in `baseline_ctr.py`. The sender uses the cryptography library to encrypt the required command, `{"action": "READ", "path": "notes.txt"}`, with AES-256-CTR. Each run generates a fresh 32-byte key and a 16-byte IV. I intentionally left out a MAC and replay protection to demonstrate the limitations of encryption alone.

The relay receives the ciphertext without receiving the key. It knows the fixed command format and the location of READ. It replaces READ with WAIT by XORing the difference between those four known plaintext bytes into the corresponding ciphertext bytes. Both actions have four bytes, so the message length stays unchanged. The receiver decrypts and parses the JSON, then logs the simulated action. It does not actually read a file or execute a command. A separate receiver is used to demonstrate replay of the original record.

Experiment Outline

The receiver first decrypted the unmodified ciphertext and logged action=READ, path=notes.txt. After the relay modified the ciphertext, the receiver logged action=WAIT, path=notes.txt without rejecting the change. The path stayed the same, and the relay did not need the encryption key.

The output showed READ as 52454144 and WAIT as 57414954 in hexadecimal. Their XOR difference was 05040810. The XOR difference between the matching original and modified ciphertext bytes was also 05040810, confirming: $C \text{ XOR } C' = P \text{ XOR } P'$

For replay, I sent the exact same original IV and ciphertext to a separate receiver twice. It logged Processed #1: action=READ, path=notes.txt and Processed #2: action=READ, path=notes.txt. Therefore, the same encrypted record was accepted and processed twice. All demonstration checks passed.

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ set -o pipefail
python baseline_ctr.py 2>&1 | tee evidence/task1-baseline.txt
DEMONSTRATION 1: ciphertext modification
Original plaintext: {"action":"READ","path":"notes.txt"}
IV: fb7f72c6fca93153a17af0650d880da5
Original ciphertext: 2866b2a7d77022d4be0e77204100330a095c605e13abeec3b899023aae262508d45d2529
Processed #1: action=READ, path=notes.txt
Modified ciphertext: 2866b2a7d77022d4be0e77254508230a095c605e13abeec3b899023aae262508d45d2529
Processed #2: action=WAIT, path=notes.txt
Modified plaintext: {"action":"WAIT","path":"notes.txt"}

XOR relation: C XOR C' = P XOR P'
READ bytes: 52454144
WAIT bytes: 57414954
Plaintext XOR: 05040810
Ciphertext XOR: 05040810
PASS: relay changed READ to WAIT without receiving the key.

DEMONSTRATION 2: replay
Processed #1: action=READ, path=notes.txt
Processed #2: action=READ, path=notes.txt
PASS: the identical IV and ciphertext were processed twice.

All Task 1 demonstration checks passed.
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$
```

What is the Root Cause?

AES-CTR encrypts by XORing plaintext with a keystream. If P is the plaintext, S is the keystream, and C is the ciphertext, then $C = P \text{ XOR } S$. Changing C by a difference D causes the decrypted plaintext to change by the same D :

$$C' = C \text{ XOR } D$$

$$P' = C' \text{ XOR } S = P \text{ XOR } D$$

Because the relay knew the original action and its location, it could choose D to change READ into WAIT without knowing S or the key. The missing MAC means the receiver has no cryptographic integrity check to detect that the ciphertext was altered. A valid JSON result is not proof of authenticity.

Replay has a separate cause: the receiver has no authenticated sequence numbers or stored replay state to distinguish a new message from a duplicate. Even an authentic encrypted record can be replayed. These results show why confidentiality must be combined with message authentication and a separate replay-protection mechanism.

Task 2 - Authenticated Finite-Field Diffie–Hellman Setup

The Design Connection

I implemented an in-process handshake between gateway-01 and node-01. Each party uses a long-term 3072-bit RSA signing key and a preconfigured trusted peer public key. Each session generates fresh ephemeral DH private values using `ffdhe3072.pem` and fresh 16-byte nonces. All cryptographic operations use maintained library implementations.

The canonical transcript contains CSCE465-HS-v2, `ffdhe3072`, both identities, both DH public values, and both nonces, with gateway fields before node fields within each pair. Every field has a four-byte big-endian length prefix, and malformed lengths are rejected before hashing. DH public values and the shared secret Z use 384-byte big-endian encoding with left zero-padding.

Each party signs `role || SHA-256(transcript)` using RSA-PSS with SHA-256 and verifies the expected peer's identity, role, and signature before accepting. DH establishes the shared secret; RSA authenticates the exchange rather than encrypting application records. The exact supplied KDF binds Z to the transcript hash and uses distinct labels to derive separate encryption and MAC keys for each direction, plus an eight-byte session identifier.

The Results

Both endpoints derived identical session material with four distinct 32-byte traffic keys. The demonstration rejected changed nonces or DH values, malformed transcripts, unexpected identities, invalid signatures, incorrect RSA public keys, and reflected messages. A second session generated fresh DH values, nonces, and a different session identifier. All demonstration checks and the 18 handshake test cases passed.

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ set -o pipefail
python handshake.py 2>&1 | tee evidence/task2-handshake.txt
RSA signing key sizes: 3072 3072
Transcript field lengths: [13, 9, 10, 7, 384, 384, 16, 16]
Transcript SHA-256: 42728f208afda27c9feae45199358cc49730cb7bb45f5c27a85693ca7a562cdf
REJECTED changed peer nonce: invalid peer signature
REJECTED changed peer DH value: invalid peer signature
REJECTED unexpected identity: unexpected peer identity
REJECTED malformed length: malformed transcript: truncated field
REJECTED invalid RSA-PSS signature: invalid peer signature
REJECTED incorrect RSA public key: invalid peer signature
REJECTED reflected handshake: unexpected peer role
PASS: both parties derived identical session material.
PASS: four distinct 32-byte directional keys.
Session 1 identifier: c3ff090be42e94be
Session 2 identifier: 88488a2d8aa3b97d
PASS: fresh DH public values and nonces in the second session.
All Task 2 demonstration checks passed.
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$
```

Concepts

Fresh signed nonces and checks against each endpoint's pending values bind the exchange to the current session, providing handshake replay resistance. Consumed state cannot be reused.

Ephemeral DH provides forward secrecy, assuming old private values are unavailable: later RSA-key compromise alone does not reveal earlier session secrets.

Signing both identities, nonces, and DH values prevents basic man-in-the-middle substitution because an attacker cannot sign altered contents under the trusted peer key. Distinct signed roles and expected-role checks prevent reflection. The KDF labels provide directional and purpose-based key separation.

Without signatures or another authentication mechanism, DH would not establish peer identity. An attacker could substitute DH values, establish separate secrets with both endpoints, and intercept or alter subsequent messages.

Task 3 - Encrypt-then-MAC Record Layer

I implemented `seal()` and `open_record()` in `secure_record.py` using the assignment's encrypt-then-MAC construction. The 15-byte header contains `version(1)`, `direction(1)`,

sequence(8), message_type(1), and ciphertext_length(4), with integers encoded big-endian. The IV is session_id(8) || sequence(8). AES-256-CTR encrypts the plaintext, and HMAC-SHA-256 authenticates header || iv || ciphertext. Each direction uses separate encryption and MAC keys established by the handshake.

Sequence numbers begin at zero independently in each direction. Preserved sender state advances the sequence without wrapping, and the 16-byte plaintext limit prevents overlapping CTR counter blocks under the prescribed IV layout. Counters must not be reset under existing session keys. The receiver expects the exact next sequence and verifies the HMAC using the library's constant-time HMAC.verify() before decryption. It also checks the authenticated version, direction, and IV. An error releases no plaintext and leaves the receive counter unchanged; successful receipt advances it.

The demonstration accepted READ notes.txt from gateway to node and OK in the reverse direction. It rejected modified headers, modified ciphertext, reflected records, replays, and out-of-order records. Consecutive gateway records had distinct IVs, and oversized plaintext was rejected without consuming a sender sequence. All Task 3 demonstration checks passed.

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ set -o pipefail
python secure_record.py 2>&1 | tee evidence/task3-records.txt
Header bytes: 15
Maximum plaintext bytes per record: 16
Initial send sequences: 0 0
REJECTED modified ciphertext: invalid MAC
REJECTED modified header: invalid MAC
REJECTED reflected record: invalid MAC
Accepted gateway-to-node: READ notes.txt
REJECTED replayed record: unexpected sequence
Accepted node-to-gateway: OK
REJECTED out-of-order record: unexpected sequence
PASS: rejected records did not advance receive state.
PASS: distinct IVs for consecutive gateway records.
REJECTED oversized plaintext: plaintext exceeds 16-byte record limit
All Task 3 demonstration checks passed.
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$
```

Task 4 - Adversarial Tests and Security Note

I implemented 42 pytest cases: 18 handshake cases and 24 record-layer cases. They cover valid handshakes and bidirectional messages, modified ciphertext and authenticated headers, replay, opposite-direction reflection, incorrect RSA keys, invalid signatures, and reflected handshake messages. Each negative test checks a specific error, such as invalid MAC, unexpected sequence, or invalid peer signature. Record rejection checks also confirm that no plaintext is returned and receive state does not advance. Temporary test guards fail if rejected records reach decryption. All 42 tests passed.

Shell

```
set -o pipefail
python -m pytest tests/ -v --tb=short 2>&1 | tee evidence/task4-pytest.txt
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 --
/home/ubuntu/csce465-agentsec/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/ubuntu/csce465-agentsec/hw2
collecting ... collected 42 items

tests/test_handshake.py::test_valid_handshake_and_key_separation PASSED [ 2%]
tests/test_handshake.py::test_exact_kdf_and_left_padding PASSED [ 4%]
tests/test_handshake.py::test_rejected_peer_handshake[nonce-invalid peer
signature] PASSED [ 7%]
tests/test_handshake.py::test_rejected_peer_handshake[public_value-invalid peer
signature] PASSED [ 9%]
tests/test_handshake.py::test_rejected_peer_handshake[identity-unexpected peer
identity] PASSED [ 11%]
tests/test_handshake.py::test_rejected_peer_handshake[signature-invalid peer
signature] PASSED [ 14%]
tests/test_handshake.py::test_rejected_peer_handshake[public_key-invalid peer
signature] PASSED [ 16%]
tests/test_handshake.py::test_rejected_peer_handshake[reflection-unexpected peer
role] PASSED [ 19%]
tests/test_handshake.py::test_rejected_peer_handshake[relabeled_reflection-invalid
peer signature] PASSED [ 21%]
tests/test_handshake.py::test_malformed_transcript_rejected_before_hashing[length-
malformed transcript: truncated field] PASSED [ 23%]
tests/test_handshake.py::test_malformed_transcript_rejected_before_hashing[truncat
ed-malformed transcript: truncated field] PASSED [ 26%]
tests/test_handshake.py::test_malformed_transcript_rejected_before_hashing[trailin
g-malformed transcript: trailing bytes] PASSED [ 28%]
tests/test_handshake.py::test_malformed_transcript_rejected_before_hashing[public_
width-DH public values must be 384 bytes] PASSED [ 30%]
```

```
tests/test_handshake.py::test_malformed_transcript_rejected_before_hashing[nonce_w  
idth-nonces must be 16 bytes] PASSED [ 33%]  
tests/test_handshake.py::test_local_session_binding[4-local DH value does not  
match this session] PASSED [ 35%]  
tests/test_handshake.py::test_local_session_binding[6-local nonce does not match  
this session] PASSED [ 38%]  
tests/test_handshake.py::test_consumed_handshake_cannot_be_reused PASSED [ 40%]  
tests/test_handshake.py::test_old_transcript_rejected_by_fresh_session PASSED [ 42%]  
tests/test_secure_record.py::test_valid_handshake_and_bidirectional_messages  
PASSED [ 45%]  
tests/test_secure_record.py::test_modified_authenticated_bytes[version] PASSED [ 47%]  
tests/test_secure_record.py::test_modified_authenticated_bytes[direction] PASSED [ 50%]  
tests/test_secure_record.py::test_modified_authenticated_bytes[sequence] PASSED [ 52%]  
tests/test_secure_record.py::test_modified_authenticated_bytes[message-type]  
PASSED [ 54%]  
tests/test_secure_record.py::test_modified_authenticated_bytes[iv] PASSED [ 57%]  
tests/test_secure_record.py::test_modified_authenticated_bytes[ciphertext] PASSED [ 59%]  
tests/test_secure_record.py::test_modified_authenticated_bytes[tag] PASSED [ 61%]  
tests/test_secure_record.py::test_malformed_record[short-record too short] PASSED [ 64%]  
tests/test_secure_record.py::test_malformed_record[truncated-record length  
mismatch] PASSED [ 66%]  
tests/test_secure_record.py::test_malformed_record[trailing-record length  
mismatch] PASSED [ 69%]  
tests/test_secure_record.py::test_malformed_record[declared_length-record length  
mismatch] PASSED [ 71%]  
tests/test_secure_record.py::test_replayed_record PASSED [ 73%]  
tests/test_secure_record.py::test_record_reflected_into_opposite_direction PASSED [ 76%]  
tests/test_secure_record.py::test_out_of_order_record PASSED [ 78%]  
tests/test_secure_record.py::test_authenticated_but_invalid_metadata[version-unsup  
ported version] PASSED [ 80%]  
tests/test_secure_record.py::test_authenticated_but_invalid_metadata[direction-wro  
ng direction] PASSED [ 83%]  
tests/test_secure_record.py::test_authenticated_but_invalid_metadata[sequence-unex  
pected sequence] PASSED [ 85%]  
tests/test_secure_record.py::test_authenticated_but_invalid_metadata[iv-IV does  
not match session and sequence] PASSED [ 88%]  
tests/test_secure_record.py::test_supported_plaintext_boundaries[] PASSED [ 90%]
```

```

tests/test_secure_record.py::test_supported_plaintext_boundaries[AAAAAAAAAAAAAAAA]
PASSED [ 92%]
tests/test_secure_record.py::test_oversized_plaintext_rejected_without_consuming_s
equence PASSED [ 95%]
tests/test_secure_record.py::test_consecutive_records_have_distinct_ivs PASSED [
97%]
tests/test_secure_record.py::test_sequence_exhaustion_fails_closed PASSED [100%]

===== 42 passed in 3.95s =====
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$

```

Security Note

The attacker is an on-path adversary who can observe, modify, inject, reorder, reflect, or replay handshake messages and application records. The attacker does not initially possess the trusted RSA private signing keys or the derived session keys. Each endpoint is configured with the expected peer identity and RSA public key. The simulation assumes a trusted local DH parameter file. Its non-goals include endpoint compromise, denial-of-service prevention, traffic-analysis resistance, concurrent access, persistent session recovery, and deciding whether an application command is appropriate. All application actions are simulated locally.

CTR encrypts plaintext by XORing it with a keystream generated from AES counter blocks. Every counter block used with a particular encryption key must be unique, including blocks across different records. Repetition exposes relationships between plaintexts. This assignment specifies an IV formed from the eight-byte session identifier and eight-byte sequence number. Consecutive sequence values are consecutive initial counters. Therefore, this implementation limits each record to sixteen plaintext bytes so a record cannot consume another record's counter block. Counters start at zero, advance without wrapping, and require a fresh handshake after exhaustion. Recreating endpoint state under existing keys is prohibited.

Encryption alone cannot detect the controlled modification shown in Task 1. The record layer instead authenticates the header, IV, and ciphertext with HMAC-SHA-256. The receiver validates basic framing, verifies the MAC using the library, and checks authenticated metadata before constructing a decryptor. This prevents unauthenticated plaintext from reaching the application and avoids using decryption behavior as feedback for invalid records. A failure returns no plaintext and does not advance the receive sequence. Test guards confirmed that rejected records did not reach the decryption entry point. Strict ordering rejects duplicates and records arriving ahead of the expected sequence.

Independent encryption and MAC keys give each key one cryptographic purpose. Direction-specific labels also separate gateway-to-node traffic from node-to-gateway traffic. Consequently, reflecting a gateway record into the gateway's receive channel fails authentication because that channel uses the other direction's MAC key. The authenticated direction field provides an additional policy check. The supplied derivation binds these keys to the transcript hash, while its session identifier binds the record IV to the established session. A shared public IV value across directions is acceptable here because the encryption keys differ.

The handshake protects key establishment using RSA-PSS signatures over a role and the transcript hash. The canonical transcript includes both identities, both ephemeral DH public values, and both fresh nonces. Each receiver checks its own contribution against its pending session and verifies the expected peer's signature. This prevents an old signed transcript from being substituted into a fresh local exchange and blocks basic reflection and man-in-the-middle attempts. Without authentication, an attacker could substitute DH values and establish separate secrets with the two endpoints. Ephemeral DH supports forward secrecy, assuming old private values remain unavailable; Python reference clearing does not guarantee secure memory erasure.

AES-GCM is an AEAD construction that combines encryption and authentication and can authenticate an unencrypted header as associated data. A maintained AEAD interface reduces the amount of composition code an application must manage. It still requires correct nonce handling, trusted key establishment, and application-level replay checks. AES-GCM was discussed rather than substituted because this assignment explicitly requires AES-CTR followed by HMAC.

Finally, a valid MAC establishes that a record was created by a holder of the appropriate session key and remained intact. It does not prove that its contents are safe. A compromised or misguided authenticated endpoint could send a dangerous tool request. The Homework 1 lesson therefore still applies: cryptographic message protection must be followed by a separate authorization boundary that checks the user's intent, allowed operations, and permitted resources before any real tool executes.

Sources

CSCE 465/765. “Homework Assignment 2: Protect Agent Messages with Classic Cryptography.” September 24, 2026. Course assignment handout.

Dworkin, M. Recommendation for Block Cipher Modes of Operation: Methods and Techniques. NIST Special Publication 800-38A, December 2001. Section 6.5 and Appendix B.
<https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38a.pdf>

Python Cryptographic Authority. “Symmetric Encryption.” Cryptography documentation. AES-CTR and GCM sections.
<https://cryptography.io/en/latest/hazmat/primitives/symmetric-encryption/>

Python Cryptographic Authority. “RSA.” Cryptography 49.0.0 documentation. Signing and verification sections.
<https://cryptography.io/en/49.0.0/hazmat/primitives/asymmetric/rsa/>

Python Cryptographic Authority. “Diffie-Hellman Key Exchange.” Cryptography documentation.
<https://cryptography.io/en/latest/hazmat/primitives/asymmetric/dh/>

Python Cryptographic Authority. “Hash-Based Message Authentication Codes (HMAC).” Cryptography 49.0.0 documentation.
<https://cryptography.io/en/49.0.0/hazmat/primitives/mac/hmac/>

pytest developers. “How to Monkeypatch/Mock Modules and Environments.” pytest documentation.
<https://docs.pytest.org/en/stable/how-to/monkeypatch.html>